

**LAB Manual – 2**

Code/Course: 23CSE312/Distributed Systems

Topic: Shared Memory Computation Model**[CO05] [BTL-3]****Student Name:** Garlapati Tejaswini**Roll No.:** AM.SC.U4CSE23219**Batch/Section:** C**Date:** 09-02-2026

1 Introduction

Shared memory communication is a fundamental computation model in which multiple concurrent processes access and update common data residing on the same physical machine. In this lab, a shared memory model is simulated in a distributed-programming setting by using Python's multiprocessing module, which creates separate process address spaces while still enabling explicitly declared shared objects to be accessed across processes. Shared variables (such as Value or Array) provide controlled shared state, allowing multiple processes to collaboratively modify and observe the same data. Since concurrent access can lead to race conditions and data corruption, synchronization mechanisms such as locks are used to enforce mutual exclusion, ensuring that shared data is updated safely and predictably.

2 Aim

To simulate a shared memory communication model on a single machine by implementing inter-process shared state in Python using the multiprocessing module and synchronization primitives.

3 Objectives

1. To create shared memory objects using multiprocessing.Value or multiprocessing.Array.
2. To spawn multiple processes that operate concurrently on shared data.
3. To demonstrate reading and writing of shared state across processes.
4. To apply synchronization (e.g., Lock) to prevent race conditions during concurrent updates.
5. (Extension) To implement shared memory with multiple data types (e.g., integer, float, and string-like data representation) and validate safe updates using locks.

4 System Requirements

- **Hardware:**

- A laptop/desktop with a multi-core CPU (recommended for observing true parallel process execution).
- Minimum 4 GB RAM (8 GB recommended for smooth execution and debugging).
- **Operating System:**
 - GNU/Linux (Ubuntu recommended) / Windows / macOS.
 - Note: On Windows, Python multiprocessing typically uses the spawn method by default; the program should be executed with a proper `if __name__ == "__main__"` guard (already standard practice).
- **Software:**
 - Python 3.8 or later.
 - Standard Python library: multiprocessing (no external packages required).
 - A code editor or IDE (e.g., VS Code / PyCharm) and terminal access.
- **Reference and Submission**
 - Submission should include labsheet document with screenshots of output for each task and the code file. Name your labsheet as *"LastFiveDigitsFromRollNumber Labsheet X.pdf"* (Eg: 23111 Labsheet 1.pdf).
 - Reference link : Multiprocessing

5 Task 1

Simulate shared memory communication models in a distributed environment. Implement a simple shared memory model in Python using multiprocessing.

Steps:

1. Create a shared memory object (using Value or Array).
2. Spawn multiple processes using multiprocessing.
3. Modify and access shared data between processes.
4. Synchronize access to the shared data, if necessary, using locks.

The multiprocessing module in Python allows you to run multiple processes in parallel on the same machine. The processes can share data in shared memory objects (like Value or Array) using locks to ensure safe access to shared data.

Shared memory is a concept that allows processes to access common data that resides in the same physical machine (laptop in this case). Even though they run concurrently, access to shared data can be controlled to prevent conflicts.

The multiprocessing module takes care of managing multiple processes on your laptop, creating separate memory spaces for each process but allowing certain variables (like the shared memory Value object) to be accessible across processes.

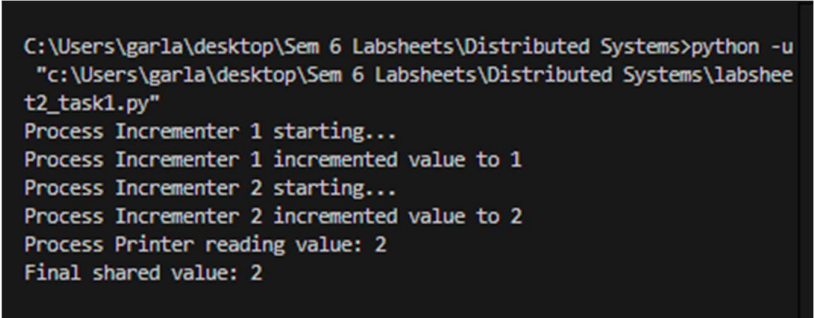
Expected Output

```
Process Incrementer 1 starting...
Process Incrementer 1 incremented value to 1
Process Incrementer 2 starting...
Process Incrementer 2 incremented value to 2
Process Printer reading value: 2
Final shared value: 2
```

Interpretation

Process 1 and Process 2 increment the shared value, and Process 3 prints it. Since the lock is used, the shared value is modified safely by one process at a time.

Results



```
C:\Users\garla\desktop\Sem 6 Labsheets\Distributed Systems>python -u
"c:\Users\garla\desktop\Sem 6 Labsheets\Distributed Systems\labsheet2_task1.py"
Process Incrementer 1 starting...
Process Incrementer 1 incremented value to 1
Process Incrementer 2 starting...
Process Incrementer 2 incremented value to 2
Process Printer reading value: 2
Final shared value: 2
```

6 Task 2: Extension — Shared Memory with Multiple Data Types

Use different shared memory objects (e.g., integers, floats, and strings) and modify them using different processes.

Steps:

1. Create a shared memory array that holds multiple data types (e.g., integers, floats, and strings).
2. Each process will modify one data type (e.g., Process 1 modifies the integer, Process 2 modifies the float, etc.).
3. Synchronize access using locks to ensure that no data corruption occurs.

Expected Output

```
Process 1 modified integer to 1
Process 2 modified float to 3.14
Process 3 modified string to 'Hello'
Final array: [1, 3.14, 'Hello']
```

Results

```
C:\Users\garla\Desktop\Sem 6 Labsheets\Distributed Systems>python -u
"c:\Users\garla\Desktop\Sem 6 Labsheets\Distributed Systems\labsheet2_task2.py"
Process 1 modified integer to 1
Process 2 modified float to 3.14
Process 3 modified string to 'Hello'
Final array: [1, 3.14, 'Hello']
```